

Supporting Systems Development by Capturing Deliberations During Requirements Engineering

Balasubramaniam Ramesh, *Member, IEEE*, and Vasant Dhar

Abstract—Support for various stakeholders involved in software projects (designers, maintenance personnel, project managers and executives, end users) can be provided by capturing the history about design decisions in the early stages of the systems development life cycle in a structured manner. Much of this knowledge, which we call the *process knowledge*, involving the deliberation on alternative requirements and design decisions, is lost in the course of designing and changing such systems. Using an empirical study of problem-solving behavior of individual and groups of information systems professionals, we have developed a conceptual model called REMAP (REpresentation and MAintenance of Process knowledge) that relates process knowledge to the objects that are created during the requirements engineering process. This model includes as part of it the Issue Based Information Systems (IBIS) method, which is designed to record the argumentation related to deliberations. We have implemented a prototype of an environment based on our model that can provide assistance to the various stakeholders involved in the design and management of large systems.

Index Terms—Process knowledge, group design, design rationale, stakeholder support, requirements engineering, argumentation, conceptual modeling, reason maintenance.

I. INTRODUCTION

THERE are a variety of stakeholders involved in large software projects, each having a different set of goals and priorities. Designers might be interested in dealing with the complexity which results from the dependencies among different parts of the problem, in communicating their decisions promptly to other designers, reusing design fragments, and so on. Maintenance personnel might be interested in knowing about the interactions between various parts of the software and the rationales underlying the design decisions. Project managers and executives might be interested in shaping design decisions from a business perspective, and as the project proceeds, in knowing what parts of it are likely to prove problematic and lead to cost and schedule overruns. End users would like to respond to design decisions which affect the product, the evolving prototypes, and to modify requirements.

Our central thesis is that for each of the stakeholders, some useful support can be provided by recording in some structured manner the history of a design. The history could encompass

the various phases of the life cycle. In the requirements analysis phase, much of it would involve discussions/deliberations aimed at polishing an initially fuzzy set of requirements into a precise one, and that of making decisions about what is to be modeled and how. In the design phase it might involve making choices among representations. In the maintenance phase it would consist of changes made to an existing system, the reasons for the changes, and the effects of such changes.

It should be noted that throughout this paper, the term *design* is used to refer to any activity which leads to the creation of artifacts. Potts and Burns [1] note that even the early phases of the systems-development life cycle involve creation of intermediate artifacts. We use the term *design* to denote these activities as well.

The REMAP project recognized the importance of capturing process knowledge to reason about the consequences of changing conditions and requirements in systems design and maintenance [2]. The most important component of this process knowledge is the knowledge about reasons behind design decisions or *design rationales* which shape the design. Our focus is on capturing history in the upstream part of the life cycle; namely, requirements engineering.

Recent studies have confirmed that capturing design rationale is especially important in large-scale development efforts for the following reasons:

- Large projects involve multiperson teams. As it is difficult to maintain multiple channels of communication in the absence of mechanisms for representing design decisions, project teams often rely on one or more team members for maintaining communication and coordination [3]. Thus large projects tend to depend on the performance of a few individuals in their (often informal) role as facilitators of coordination and communication.
- In large projects, over time and with changing development groups, the rationales and context for key design decisions are often confused and even lost [3]. This is largely a result of the inadequacies of current documentation practices in capturing the knowledge about design rationales and decisions.
- Critical errors are commonly made in the formulation and resolution of design decisions, and these are often unnoticed, since they are not recorded [4], [5].
- Different project teams are involved in different phases of systems development. In the absence of knowledge about design rationales, key design decisions are often misunderstood and misinterpreted [6]. This problem is

Manuscript received October 1, 1991; revised February 12, 1992. Recommended by M. Jarke and A. Borgida. This work was supported in part by the National Science Foundation through Grant 8713684, and support for domain analysis was provided by the Software Technology Program of the Microelectronic Computer Technology Corporation (MCC) and Andersen Consulting. ConceptBase was provided by the University of Passau.

B. Ramesh is with the Naval Postgraduate School, Monterey, CA 93943.

V. Dhar is with New York University, New York, NY 10003.

IEEE Log Number 9200170.

especially serious during the maintenance phase of the life cycle.

The long-term goal of our research is the development of a generic model for capturing and representing process knowledge (design rationales and their linkages to design artifacts) across various phases of systems development. In this research we have developed a conceptual model to characterize a task in the early stages of systems development—the task of requirements engineering.

There are several reasons for focusing on the upstream portion of the life cycle:

- Research on knowledge based software engineering has recognized that the maintenance of software should be done at the level of specifications, rather than at the level of code to achieve high-quality software [7]. Specifications are the closest in form to the user's conceptual model of the system, the least complex, and most localized [8]. Even simple modifications at the specification level can lead to major changes at the implementation level. Therefore Balzer *et al.* [7] have proposed a software-development paradigm in which maintenance is done at the level of specifications, and the changed specifications are transformed into optimized implementations. Accordingly, we view the process of maintenance as one of redesign at the level of specifications.
- One of the objectives of our research is to provide a framework for developing an ontology of the application domain in terms of the knowledge used in creating systems in that domain. Potential for reuse of this knowledge in designing and maintaining systems in the domain provides the major motivation for developing such an ontology. Based on experiences in several design situations, Biggerstaff and Richter [9] conclude that from the view point of reusability, reuse of components at the design stage has more potential for payoff than reuse of code, and that design reuse is the only way to achieve orders-of-magnitude increase in software-development productivity. Experiences with reusability at the level of designs [10] suggest that in order to achieve high reuse, reusable components should be specified at the level of *what* is to be done rather than *how* it is done. This recommendation corresponds to describing reusable components at the level of specifications.

The challenge in this research is to represent the relevant parts of the history in a way which supports the various stakeholders. How this is done is described in Section IV. First, we provide a description of an empirical study of the requirements engineering process that shaped significantly our conceptual model. This is followed by an explanation of the model. The implementation and knowledge representation language are described in Section III.

II. MODEL DEVELOPMENT STUDY

The conceptual model of a design history consists of primitives or semantic elements which characterize the variety of

problem solving and communication that can take place in the domain of interest. How does one determine the appropriate set of primitives?

Our approach to this has been an empirical one. We began with a set of primitives from the literature. These were then augmented on the basis of the empirical study. Specifically, we observed individual and groups of experienced systems analysts go through a requirements engineering exercise with a case which had been previously developed by one of the authors. By virtue of this experience, this researcher was an expert on the case, which was essential in order to play a good user role in the study which we describe shortly. The case was based on a large real-world system developed for clients in the utilities industry. The domain analysis that aided the development of the case study is described in [11].

A. The Initial Primitives

Programming in the large (large software-development efforts) is characterized by project teams with thin distribution of knowledge among members of the team [3]. As these projects involve often large and complex problems, creation of design solutions involves knowledge that spans several areas, including the application domain, system architecture, machine architecture, and algorithm structure [3]. Since no single designer possesses all the knowledge required to produce a solution, a team of several members is typically involved in a design task. Software design is essentially a cooperative task and requires information exchange that distributes various relevant components of knowledge among the group members. Further, the task involves resolution of underlying incompatibilities among *mental models* of individual team members to achieve a consensus view necessary for productive work [12], [13]. Recent research on the group software-design process also emphasizes the need for mechanisms to facilitate assumption surfacing and knowledge exchange [14]. We regard conflicts as normal aspects of the dialectic process through which team members share knowledge.

The Design Journal research project at the Microelectronic Computertechnology Corporation (MCC) identified the Issue-Based Information Systems method (IBIS) [15] as a useful tool for representing design deliberations in large design projects [16]. The IBIS method provides a rhetorical model for representing "argumentation" processes [17]. This approach is similar in spirit to the philosophy behind communication tools such as the COORDINATOR [18] that characterize conversations in terms of speech acts. Our strategy was to use concepts from IBIS and speech acts in order to interpret our case analyses. Additional primitives were introduced in order to interpret the unexplained parts of the case analyses.

The following primitives comprised the initial set. Concepts are in regular type and the relationships among them are italicized:

- issue: a problem, concern, or question that requires discussion for the problem solving to proceed [15].
- position: a statement or assertion that *responds* to an issue. Positions need not be mutually exclusive of each other.

Each issue can have one or more positions.

- *argument*: arguments are statements that *support* or *object* to positions. Each position may have one or more arguments that either *support* or *object* to it.

The deliberation process is modeled by a network consisting of issues, positions, and arguments. The relationships between issues, positions, and arguments are denoted by the links between the nodes.

The IBIS model does not recognize the context in which argumentations occur, nor the outcomes of the argumentation. For instance, Lee [19] cites the absence of explicit representation of *goals* which drive the process of argumentation. The need for using a task-oriented model of the group process is emphasized by Hahn *et al.* [20]. As the primary focus of our research is the requirements engineering task, requirements and design objects representing the inputs and outputs to the process were used as additional components of this initial model. Specifically, the following additional components comprised the initial model:

- *requirements*: a capability needed by a user to solve a problem or achieve an objective. A capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document. The set of all requirements that form the basis for subsequent development of the software or software component [21].
- *design objects*: part of design solution that satisfies a requirement.

B. Study Design

There are two types of activities involved in group design efforts. First, members of a team might independently work-out partial solutions to small problems or identify issues or obstacles which need to be addressed in solving these problems (this can, of course, also occur in a group setting). Secondly, members share and communicate solutions and issues in a group context.

We collected data in both contexts. Think-aloud protocols of problem-solving behavior of a few subjects engaged in a simulated requirements engineering task were collected. The second source of data was from group meetings of analysts engaged in a simulated requirements engineering exercise. The primary source of data for this phase of our research was transcripts of group meetings in which analysts were engaged in putting together a common solution for a specific task. The experimental task consisted of a case study involving the development of functional specifications for a transaction-processing system. The case study describes a representative design task in an application domain.

1) *Subjects*: Practicing information systems (IS) professionals and advanced students of systems analysis and design were used as subjects in the study. Our subjects had experience ranging from 5 to over 30 years in information systems development.

The subjects were drawn primarily from a continuing education program at New York University designed for advanced

information systems professionals interested in acquiring and evolving their skills in developing large software systems.

The verbal protocol analysis was performed with four subjects. All these were practicing IS professionals. They had an average of 9 years experience in IS development, of which over 4 years was in systems analysis and design activities. A total of seven groups, consisting of groups of two or three, participated in the study. These subjects had an average of 13 years experience in IS development, of which over 10 years was in systems analysis and design activities.

2) *The Task*: The experimental task was in the area of developing functional specifications based on a case study. It described a scenario in which the initial requirements of a customer order processing system were specified.

The subjects were given the case containing informal description of user requirements. The subjects were required to develop a functional specification of a system (in the form of DFD's) to satisfy the "user requirements." As the primary focus of our study was to characterize the nature of design rationales in developing design solutions from requirements, the subjects were asked to clearly articulate decisions made and the reasons for making such decisions.

Individual subjects in the empirical study were asked to think aloud, verbalizing their thoughts as they carried out their task of developing functional specifications. These subjects were given a warm-up session to practice thinking aloud.

The subjects in the group study were not instructed to think aloud. Our objective was to develop primitives necessary to characterize the nature of discussions in group settings. The subjects had worked on developing individual solutions before the group meetings, and the nature of the group decision-making session was one of communicating *individual* solutions to other team members and arriving at a common solution. The very nature of the task necessitated communication among group members by verbalization.

The subjects were given opportunities to seek clarifications on informal specifications by participating in client meetings. In fact, this was a major component of the exercise. The subjects could ask specific questions they may have had for their various clients. These questions were directed at different types of clients they would have dealt with in a real-life situation. Given that the researcher had put in extensive effort in the development of the case [11], it was natural for him to play the roles of the various clients and provide answers to specific questions. He provided only requested clarifications and did not attempt to probe the reasons for specific behavior. Each subject and team member participated in four such client meeting sessions, lasting between 30 minutes to an hour. The oral responses provided to the subjects were recorded and followed up by written responses when necessary.

3) *Data Analysis*: Design activities that the individual members had engaged in prior to group meetings were not part of our data for analysis. Only meetings in which the team as whole was working on arriving at a group solution were recorded. The meetings lasted about two hours on average.

Transcripts were segmented and labeled to identify the subject, and sequentially numbered to facilitate referencing. The initial model primitives as defined in the previous subsection

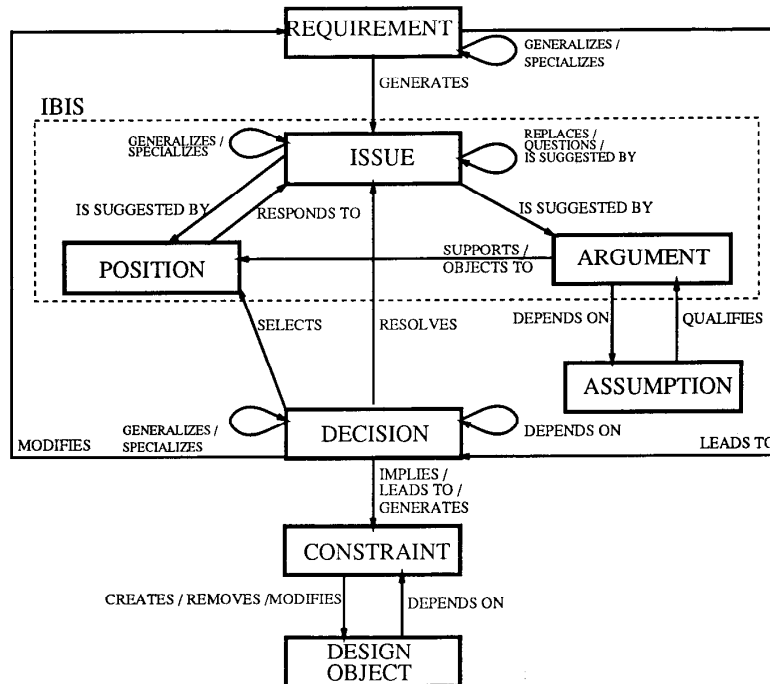


Fig. 1. Conceptual model.

were given to the coders initially to code the transcripts. These did not characterize completely the data in the initial cases. Additional concepts or relationships were introduced to account for the unexplained data, resulting in a new coding scheme. A different set of coders was used to code the transcripts with the revised coding scheme. The final coding by two independent coders was used to ascertain the reliability of the coding scheme. The final set of primitives is presented shortly.

Agreement between independent coders of protocols is considered a measure of the reliability of the coding. In our study, the reliability is measured by the nominal scale response agreement [22]–[24] among coders. The method involves constructing an agreement matrix in the form of a contingency table that categorizes the responses of two coders to the same set of objects. This matrix is then used in computing percentage agreements. Table I provides a summary of the reliability statistics in terms of percentage agreements. The percentage agreement value is considered reasonably high and is comparable to those obtained in other studies [23]–[25]. Further, the low variance between coders across subjects can be taken as an indicator of high model reliability. Thus the outcome of the empirical study was a model with primitives rich enough to characterize the use of various knowledge components used by designers in a design task. The model is presented in the next section.

C. Conceptual Model

Fig. 1 shows the derived primitives of our conceptual model based on an analysis of the data. The argumentation related to

TABLE I
SUMMARY OF INTERCODER AGREEMENT STATISTICS

Average percentage agreement	85.85
Standard deviation	4.53

design deliberations can be represented using the original IBIS primitives (within the dotted segment in Fig. 1; adapted from [15]). As is apparent, the model includes additional concepts and relationships not present in the original model. These are described below.

The model can be interpreted as follows. The design process is one of satisfying *user requirements*. The design deliberation process involves discussion and resolution of issues which are generated and must be addressed in order to satisfy requirements. Thus the requirements represent the *goals* to be satisfied by the design process. They require decisions which satisfy these goals. Initial requirements get refined, modified, and elaborated during the deliberation process, generating subgoals or issues to be resolved. In effect, the requirements provide the context in which design deliberations occur. Further, the requirements guide the design process by providing a basis for ordering (ranking) the issues to be resolved. Thus the explicit representation of requirements is a very important part of the process. Essentially, it addresses the questions: *where do the issues come from?* and, *which of the issues are more important than the other given the requirements?* The latter involves making explicit the criteria on which the issues are ranked.

In our model, a *decision* explicitly represents the resolution of issues (see Fig. 1). A decision can resolve one or more issues. A decision can also be dependent on other decisions which have been made earlier. A decision resolves an issue by selecting a position from among a set of *allowable* positions that could resolve the issue. The explicit representation of decisions allows for various evaluation criteria that could be used in resolving issues and the ability to analyze alternative designs based on different evaluation criteria. Resolution of an issue (or set of issues) during the design process (by making decisions) may also lead to modified requirements, characterizing the evolving nature of the design process.

Our model also represents the context in which arguments support positions. In the original IBIS model, if there are assumptions underlying the arguments, they are embedded in the argument, as opposed to being explicitly represented. Explicit representation of assumptions allows for mechanisms to evaluate the applicability of the arguments in a given situation, and thus helps in the evaluation of positions.

Finally, while the IBIS method provides primitives for capturing process knowledge, it does not provide primitives to relate the process knowledge to the artifacts that result from the process. In our model, we explicitly represent the relationship between resolved issues and the design solutions which are generated by such resolution. The process of resolving issues (by identifying one or more solutions) leads to commitments designers make that need to be satisfied in developing a design solution. These commitments can be thought of as imposing constraints on the design. Thus the linkage between the deliberation process and the creation of artifacts is represented by the *constraints* that are *implied* or *generated* or *led to* by *decisions* that resolve issues (as shown in Fig. 1).

The constraint could be a formal specification as in first-order logic, or a written description that the system must satisfy. Likewise, the design object could have a formal or informal semantics. It could be a dataflow diagram component such as a process, or a structured object in an object-oriented language with a well understood behavior.

Summary: As is apparent from the model in Fig. 1, requirements engineering is an iterative exercise; often, the purpose of going through the exercise is to define the problem itself, and in the process, refine requirements. That is, an initially fuzzy requirement gets clarified through discussion, leading to other, more specific requirements. While the model has been motivated by requirements engineering, its components are generic enough to be applicable across various phases of the life cycle. In other words, while *requirements* and *design objects* can be considered to represent the inputs and outputs, respectively, in the requirements engineering phase, they can be substituted with appropriate inputs and outputs for different phases of the life cycle.

Finally, the representation of the components of the model such as requirements, constraints, and design objects will depend on the design methodology used and the phase of the life cycle. The model can be used with any design representation.

III. IMPLEMENTATION AND REPRESENTATION LANGUAGE

We have developed a prototype environment to support the tasks of requirements engineering. This environment is based on our conceptual model and is intended to facilitate acquiring, maintaining, and using various components of knowledge that are used to support various stakeholders.

The implementation of the prototype is based on ConceptBase, an implementation of the high-level conceptual modeling language Telos [26] developed at the University of Passau [27], augmented with reason maintenance system functionalities. ConceptBase is organized in a client-server architecture in which clients and servers are distributed over local or wide-area networks. This facilitates the interactions among design team members using different workstations distributed across a network, accessing and manipulating a central knowledge base.

A. Knowledge Representation Language

Our model provides primitives to capture the knowledge about refinement, elaboration, and modification of initial requirements which lead to design solutions. Capturing and reasoning with the knowledge about design rationale (a form of design history) requires a conceptual modeling language for representing this knowledge, and a mechanism for management of a large knowledge base populated with such knowledge. We view the process of requirements engineering as a knowledge base management activity, wherein the knowledge about design rationale is maintained and reasoned with. This task essentially requires facilities for constructing, querying, and maintaining structured knowledge bases.

To support the representation and reasoning with process knowledge, the conceptual modeling language should be capable of representing the ontology of design rationale in terms of our model primitives and provide a mechanism to populate this metamodel with specific instances. The language should also provide automatic inferencing to enable access to the knowledge implicit in the model and provide mechanisms to maintain the integrity of the knowledge base made up of interconnected components that are incrementally modified.

The Telos language that is used in our research, was inspired by RML [28], a language developed for requirements modeling. Telos provides facilities for describing metaconcepts which allow the definition of arbitrary metamodels (such as our conceptual model) and their instantiations. An important feature of Telos that distinguishes it from most conceptual modeling languages is its uniform treatment of individuals (objects) as well as attributes; i.e., attributes are treated as first-class objects. With this powerful feature, even attributes can be instantiated, specialized, and have attributes of their own. This feature facilitates reasoning with the knowledge about the characteristics of relationships that are central to our model.

For instance, in our model in Fig. 1, ASSUMPTION qualifies ARGUMENT. This relationship can be represented as an attribute of ASSUMPTION. Further, we might want to represent characteristics of this attribute such as the strength of the qualification, the person that assigned it, the reason for the assignment, etc. In the following Telos specification,

ASSUMPTION class is defined with *qualifies* as an attribute. Further, the attribute *qualifies* is treated as a first-class object with attributes of its own:

```
Individual ASSUMPTION in Class with
attribute
    qualifies: ARGUMENT
END

Attribute ASSUMPTION!qualifies in
    Class with
attribute
    strength: Integer;
    assigned_by: Person;
    reason_for_assignment: String
END
```

Further, Telos provides aggregation, classification, and generalization mechanisms that are important features of any object-centered representation of knowledge bases to be instantiated by a metamodel such as ours.

An important component of knowledge about design rationale is its history though time. In the context of systems development, the temporal information may often be incomplete. Telos provides mechanisms for representing and reasoning with temporal knowledge through a representation proposed by Allen [29], and provides the functionality to represent incomplete information as well. Further, it represents *belief time*, the time interval associated with propositions, and believes them unless explicitly required to revise such beliefs. This facility can be used, for instance, to trace the history of assumptions which are associated with an issue and the resulting changes to design decisions and design solutions.

Further, *validity time*, which is the duration during which an individual or attribute is valid, can also be specified by the users. The semantics of this temporal information can be specified by the users using rules and constraints (which are discussed later in this section). The temporal knowledge is useful in providing support in several areas, including project management, design replay, and maintenance (see Section IV).

Telos offers a first-order assertion language for specifying integrity constraints and rules. The integrity constraints are necessary in ascertaining the semantic integrity of the design-rationale knowledge base as it is incrementally modified. Using a powerful first-order assertion language, with quantification over classes, arbitrary constraints can be specified to incorporate domain-specific knowledge and design methodology knowledge.

As an example, we may want to specify a piece of domain knowledge that states that if two decisions exclude each other, then both cannot have valid beliefs. In the following Telos descriptions, DECISION has the following attributes: excludes, validbelief, selects, and resolves. Further, Attrvalue is a four-place predicate whose arguments are the object, the attribute name, the attribute value, and validity time; the predicate NE is "not equal." Here, the constraint *onedecision* defined over the DECISION object states that if there are two decisions d and e with valid beliefs r and s, respectively, and

if d excludes e, then r and s cannot be equal. If the validbelief is assigned a value only when a decision has a valid support, this constraint will be violated only when both decisions have valid beliefs. This constraint can be further refined to include temporal intervals during which a violation occurs.

```
Individual DECISION with
constraint
    onedecision: $
    forall d/DECISION
    forall e/DECISION
    forall r/Beliefstatus
    forall s/Beliefstatus
    (AttrValue(d,validbelief,r,Always) and
    AttrValue(e,validbelief,s,Always) and
    AttrValue(d,excludes,e,Always)
    ==> NE(r,s) $
END
```

Deductive rules can be used for modifying and inferring values of attributes. The deductive rules are also important in specifying the semantics of the relationships between various components of our model. For instance, from Fig. 1, if a decision selects a position, and that position addresses an issue, then the decision resolves the issue. The *resolve* rule below specifies this relationship:

```
Individual DECISION with
rule
    resolve: $
    forall d/DECISION
    forall p/POSITION
    ((exists i/ISSUE
    AttrValue(d,selects,p,Always) and
    AttrValue(p,respondsto,i,Always)
    )
    ==> AttrValue(d,resolves,i,Always)
    ) $
END
```

The first-order assertion sublanguage, the first-class status of attributes, and the availability of metaclasses make Telos an easily extendable language. In the context of our model, we have simulated the functionalities of a special purpose Reason Maintenance System (RMS) with these features. The procedure for computing the status (label) of a datum is defined using deductive rules. The functionality of contradiction detection in the RMS is achieved by defining integrity constraints that specify contradictions. The *onedecision* constraint specified earlier can be thought of as an example of such a contradiction specification. The ability to define rules and constraints over classes of objects as well as instances provides much flexibility in defining and maintaining the dependencies. Finally, a powerful predicative language for querying the knowledge base is also provided.

IV. SUPPORT FOR REQUIREMENTS ENGINEERING

Our conceptual model has been defined in terms of object classes corresponding to objects and relations in Fig. 1. Our REMAP design-support environment provides mechanisms

TABLE II
SHAPES AND SHADES OF PRIMITIVE TYPES

Primitive Type	Shape	Color
requirement	circle	white
issue	ellipse	white
position	rounded box	white
argument	diamond	dark grey
assumption	box	white
decision	box	light grey
constraint	ellipse	dark grey
design object	rounded box	dark grey

to instantiate, query, and modify instances of these objects interactively, thereby facilitating the incremental acquisition of process knowledge and application domain knowledge. Further, a hypertext-like browsing capability to view instantiated objects facilitates convenient traversal of the knowledge base.

A. Capturing Design Rationale

REMAP supports the capture of design rationale knowledge by providing a mechanism for design teams to conduct their deliberations using the primitives of our model. As the design deliberation proceeds, various issues, positions, arguments, assumptions, decisions, etc., are defined by the users.

Fig. 2 illustrates the results of a design deliberation session. The example is drawn in part from a design deliberation recorded during our model development study. A design team engaged in the deliberation of the requirement *process_order* identifies two issues that need to be resolved. One issue concerns the priority scheme to be used in grouping orders for processing, and the other concerns the frequency of processing. Several alternatives (i.e., *positions*) for resolving the former issue *priority_scheme* are considered. The *Process Model* window in Fig. 2 shows that *location*, *customer_type* and *service_type* are the priority schemes that respond to the issue. Each of these positions has different supporting or objecting arguments, each of which in turn is supported by assumptions. For instance, the position *customer_type* is supported by the argument that this prioritization scheme will facilitate the providing of *prompt_service* to customers. This argument is based on the assumption that a *versatile_crew* is available to handle any type of request from a customer. Issues are resolved by making a decision to select a position, thereby leading to a constraint that needs to be satisfied by design objects. In our example, a decision *processing_orientation* selects the position *service_type*, leading to the constraint *processing_priority* that creates the design object *process_set_processing_priority*.

In Fig. 2, instances of different primitives in our model are displayed as icons with different shapes, and relationships are displayed as links. Table II provides the shapes and shades of different primitive types in Figs. 2 and 3 when they have valid justifications (as in a Reason Maintenance System). The objects are shaded black when they have no valid justification.

Associated with each instance is a distinct menu of choices that is defined for its class. These menu choices, in effect, define the *legal moves* possible during the deliberation process. In Fig. 2, the menu choices associated with *process_set_processing_priority*, which is an instance of *DESIGN_OBJECT*, are shown.

REMAP provides a textual view of the knowledge base with a frame-based representation as in the *Editor* window of Fig. 2. A hypertext-like view of the knowledge base is illustrated in the *Process Model* window.

As the knowledge base evolves over time, with interactive updates and modifications made by the users, semantic integrity of the knowledge base is maintained by defining appropriate integrity constraints.

Deductive rules defined over individuals or attributes or their instances provide access to inferred knowledge. In our example, the *Editor* window shows the *trace* rule that states that if a decision resolves an issue and the issue is generated by a requirement, then the requirement leads to the decision. This rule will be helpful in establishing the traceability of design artifacts to requirements. As arbitrary chaining of rules is possible, with the conclusion part of a rule becoming part of the condition part of another, interesting inferences can be drawn. The *trace* rule and the *resolve* rule discussed in Section III-A illustrate this functionality.

B. Reuse of Design Knowledge Components

If the design solution is formed by putting together components from a library, the constraints generated during design deliberations help restrict their selection. For instance, a constraint for the selection of a design object may restrict the functionality and environment [30] or the domain characteristics [31] of a design object. The availability of a powerful predicative language [32] to express semantic integrity constraints, deductive rules, and queries alleviates an important problem of selecting reusable components from the library.

Lubars [33] emphasizes the need for tailoring a generic reusable object using situation specific requirements. Our model facilitates this process by providing a very rich context (in the form of design deliberations) in which the specification of the component and its characteristics may be made. In essence, the process knowledge can be valuable to designers in developing such a specification.

C. Domain Knowledge Acquisition

If the library of components does not contain the design objects which are essential to form a complete solution, the user may define such objects. Thus a user may interactively query the knowledge base about components available in the library, and may modify or augment it. The design objects that are defined by the designer can be thought of as application domain-specific components that can be put together to form design solutions. REMAP provides a mechanism for incrementally acquiring application domain knowledge (in terms of defining a library of design objects) as the design progresses. For example, in the DFD representation,

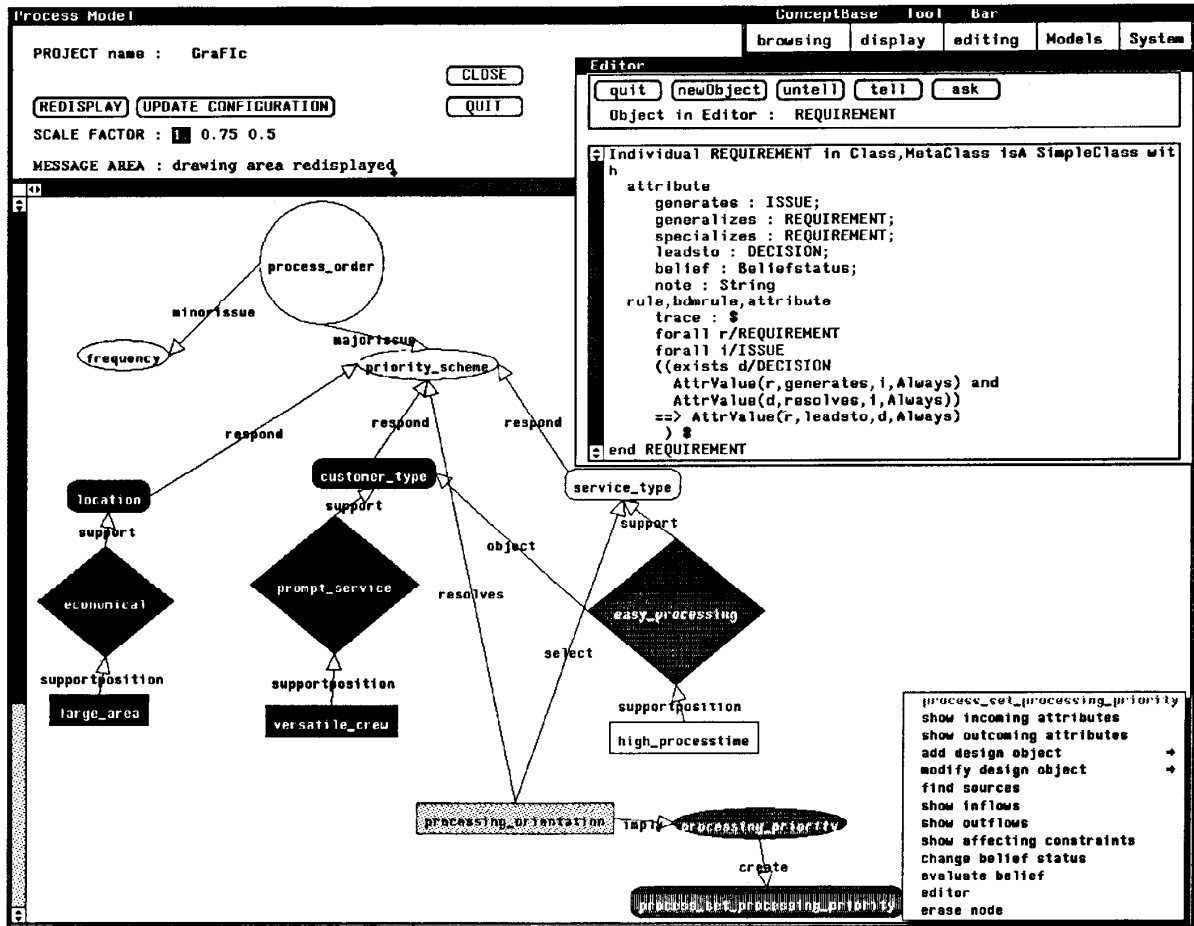


Fig. 2. Design deliberation instance.

the application domain knowledge library will consist of four types of design objects, each corresponding to one type of object in a DFD. They are: *entities*, *processes*, *stores*, and *flows*.

D. Design Synthesis

REMAP supports the design activity by providing a mechanism for synthesizing design solutions by putting together components defined by the user or drawn from a library. For example, in the DFD representation of requirements specification, the process of design synthesis involves identifying design objects (i.e., primitives such as processes, stores, entities) that become part of a design solution, and establishing

connectivity among them. Our approach is similar in spirit to that of the Programmer's/Requirements Apprentice project [34], [35], where constraints are used to restrict clitches to specific roles. Our work is uniquely distinguished from this research, as well as other research on automatic programming [8], [36] and knowledge-based requirements assistance [37], in that we use process knowledge to identify, refine, or define components that are synthesized together.

The process of putting together components to form a complete solution is done in the context of a design methodology. Valid structural relationships among the design objects and heuristics for *good* systems design are part of the knowledge about design methodology that will be useful in synthesizing

Individual DATA_STORE with
constraint:

```
Necessariesink: $ forall d/DATA_STORE
                  exists a/Attribute AttrId(d,sink,a,Always) $
Necessariesource: $ forall d/DATA_STORE
                   exists a/Attribute AttrId(d,source,a,Always) $
```

END

design solutions. This knowledge about the methodology is represented as rules or constraints over design objects. For instance, the *Necessariesink* (or *Necessariesource*) constraint specifies that each *DATA_STORE* object should have a sink (or source) attribute. Any instance of the *DATA_STORE* object that violates such a constraint cannot be instantiated in the knowledge base. *AttrId* is a four-place predicate whose arguments are the object, the attribute class, the attribute name, and validity time (see next page).

E. Design Decision Support

REMAP explicitly identifies the dependencies among its primitives and provides mechanisms for maintaining and reasoning with these dependencies.

In our model, arguments depend on assumptions. The belief status of an argument will depend on the belief status of assumptions which qualify it. Similarly, the belief status of a position depends on the belief status of the arguments which support or object to it. A dependency network can be specified using rules defined over instances or object classes. For instance, the *beliefrule2* in the *Editor* window of Fig. 3 states that an argument has the same belief status as the assumption which qualifies it.

Our reason maintenance system propagates the belief status of assumptions to arguments, and then to positions. This provides a means for identifying *allowable* positions (i.e., those that are supported by current set of beliefs in relevant assumptions). In Figs. 2 and 3 the nodes that do not have a valid justification are shaded in black. In Fig. 2, the assumption *high_processtime* supporting the argument *easy_processing* has been ascertained as valid by the designer. This leads to a valid justification for the position *service_type*, and the decision *processing_orientation* which selects it. The facility for identifying an *allowable* position can be very valuable in making a decision about selecting a position from among a set. For instance, if there is only one position that is *allowable* (among a set that addresses an issue), the obvious decision would be to select it. Additional functionalities such as multicriteria decision-support models, ranking, and voting mechanisms to support the design decision-making process can be added to further aid the selection of a position from among a set of alternatives.

In REMAP, the dependency network may include dependencies of constraints on design decisions as well as dependencies of design artifacts on constraints. With such an extended network, the belief status of a set of assumptions can be propagated to a set of design objects. In Fig. 2, the constraint *processing_priority* and design object *process_set_processing_priority* get valid justifications by the presence of a valid justification for the assumption *high_processtime*. This mechanism for identifying the design objects which have valid justifications will be very helpful to designers in determining whether a component can become part of a design solution. Also, the designer can trace the repercussions of the set of assumptions on the design solutions, or identify changes in the set of valid assumptions that can provide valid support for a design object.

F. Maintenance of Changing Requirements

Changing requirements and assumptions necessitate changes to software systems. In our model, changes to design rationale will automatically trigger changes in the belief status of design solutions, thereby suggesting redesign [2]. Since various components of the process knowledge that lead to the design solution are tightly related, changes to the constraint set resulting out of changed assumptions, decisions, or requirements will initiate the synthesis of a new design solution. Thus the process of maintenance will occur as changes in the process knowledge.

Fig. 3 shows how changes made with our tool to the process knowledge modify design solutions. The *Process Model* windows of Figs. 2 and 3 highlight the changes. In Fig. 3, the assumption, *high_processtime* supporting the argument *easy_processing* has been ascertained as invalid by the designer. This results in the position *service_type* being retracted from the set of *allowable* positions, leading to loss of support for the design object *process_set_processing_priority*. The designer has further provided a valid justification for the assumption *versatile_crew* that leads to a valid support for the position *customer_type*. With the instantiation of the decision *service_orientation*, the design object *process_set_service_priority* gets its support.

G. Reuse of Process Knowledge

Our enhanced design representation captures the rationales such that these may be used in conjunction with application-domain knowledge to modify the design incrementally. We further provide mechanisms for reuse of the process knowledge. As the dependencies among process knowledge and design artifacts are maintained by a reason maintenance system, a future revision in the belief status of relevant assumptions will automatically be propagated into changes in status of design solutions. For instance, if the assumption *high_processtime* were to become valid again in a future design situation, the system will be able to automatically instantiate the relevant decisions, constraints, and finally the *process_set_processing_priority* object in the design space.

H. Design Replay

During various phases of the life cycle, facilities to retrace the progression of steps that the design process went through can be beneficial. Such a facility is essential if corrective action is to be taken to redo parts of the design process that may have led to dead ends or undesirable solutions. Further, design history information is useful in *replaying* the steps to facilitate understanding the *evolution* of the system as well as identifying the *choice points* where alternative decisions could lead to different solution paths. In the transformational paradigm for design, design history is maintained in terms of the states that the system is at each stage and the transformations that are applied at that state to generate the next state [38].

In our representation of design history, we maintain the belief times of assertions in the knowledge base. For each assertion in the knowledge base, information on the time period during which it was believed (i.e., it was a part of

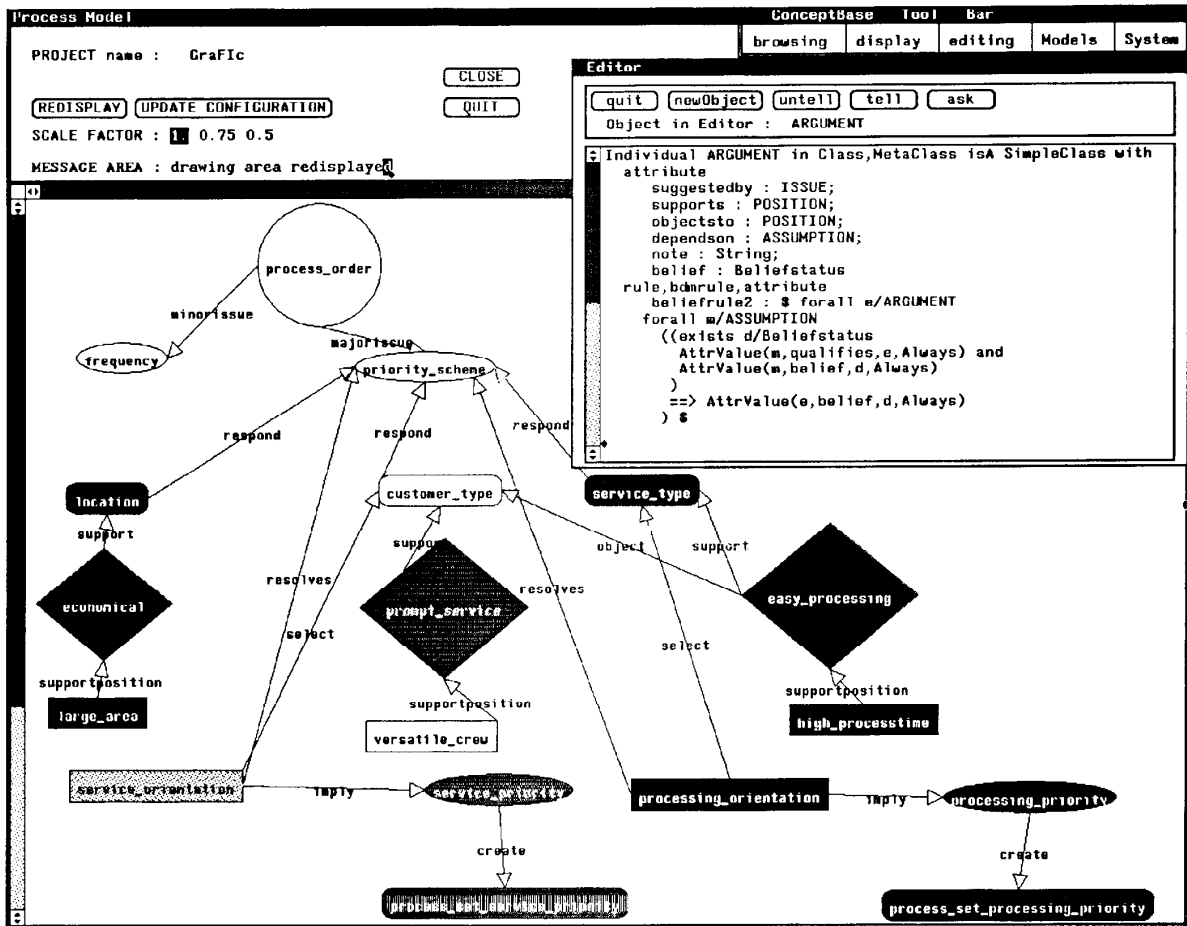


Fig. 3. Maintenance of process knowledge.

the current knowledge base) is maintained. This information can be used to trace the steps in the evolution of the design history or *design replay*. It can also be used to categorize decisions, assumptions, etc., according to the phase of the development cycle. During design replay, additional positions which resolve an issue could be defined, leading to a different set of alternatives to be considered. Also, re-evaluation of the already-defined positions based on different criteria may lead to different decisions.

Design replay can be chronological or dependency directed. The dependency information maintained by REMAP can be used to identify the decisions, and in turn the issues which lead to the creation of a specific design object. Then the issue could be re-examined to generate alternate solutions. Similarly, using the dependency-directed backtracking, the set of assumptions that a design object depends on can be identified. The assumptions could be re-evaluated to revise beliefs in them. Such a revision of beliefs could lead to a different design solution.

1. Temporal Reasoning

Facilities to represent and reason with temporal information

about individuals and attributes is very useful in an environment that is concerned with the history of the process.

For example, in Telos, the following specification of a requirement *process_order* states that the issue *frequency* has a validity time associated with it:

```

Individual process_order with
  attribute
    generates: frequency at < time-relation>
      < time-interval>
  END

```

The time relation can take any of the following forms to represent a definite or an open time interval: before, after, during, starts, finishes, contains, equal, overlaps, etc. The time interval can either be user defined or from among the following predefined: year(y), month(y,m), day(y,m,d), minute(y,m,d,h,m), always, etc.

Temporal information can be used in a variety of ways. For example, the validity time of an issue (such as *frequency* as above) could be interpreted as the time frame during which that issue must be resolved. Then this information can be used

for generating reminders to the designers or managers to focus their attention on issues which may have to be resolved within a time frame or used in rank-ordering issues.

Temporal information can also be used in inferring implied knowledge using deductive rules. For instance, an issue that has a validity time *before* a certain date may be considered to be an important one. The temporal information can also be used in defining integrity constraints. Project planning and control can be facilitated by an integrity constraint that uses validity time. For instance, the user may specify that no issue with a validity time value t_2 be resolved before all issues with validity time value t_1 are resolved, when t_2 is *after* t_1 . Such an integrity constraint might be useful for project-management purposes by providing constraints on the order in which issues are resolved. A similar constraint could be defined to ensure that before a specific time interval elapses all the issues that have been assigned, a validity time *contained* by it must be resolved. Beyond that time, the user may be required to redefine the validity time associated with the issue.

J. Summary

Much of REMAP's functionality is directed toward designers and maintainers of large systems. Design support is provided via the communication of requirements, issues, decisions, constraints, etc., among the design group. In addition, the model supports reuse at a high level and can incorporate knowledge about the design methodology (mostly as constraints) which enables the synthesis of correct designs. By providing a complete design history, the understandability of designs is greatly enhanced. Further, we provide a mechanism by which changes in design decisions can be propagated into changes in design solutions. This mechanism will be a valuable tool in iterative design development situations. Further, we can help maintain the one-to-one correspondence between requirements and design solutions. The process of maintenance at the level of requirements and design rationales will, while involving a cost of its own, significantly reduce the system-maintenance efforts.

The model can also be used by end users interested in understanding how exactly the design-deliberation process addresses their requirements. It can also be used to seek clarification of changes requested by them and to see the repercussions of requirements at the design level.

Finally, the model can provide useful information to project managers and higher level executives. First, several types of statistics can be generated from the history, such as the time taken for issue resolution, the number and frequency of changes in requirements and design decisions, their impact on the schedule, and so on. Secondly, the positions generated can be evaluated not just from a technical perspective (which is what designers are generally interested in), but from the perspective of the goals of the organization. For example, in an insurance company that among other things processes client claims, the medium of storage of data is likely to have a significant impact from a business standpoint in terms of the speed of claims processing, the error rates, fraud detection, and so on. In other words, positions, arguments, and assumptions

can be linked directly to business criteria which are of interest to high-level executives. Specifically, positions and decisions can be justified on the basis of these (nonfunctional) criteria. Finally, the history can be analyzed statistically and used to answer logical queries, which is useful from a management control perspective.

V. RELATED WORK

It is worth contrasting REMAP with other models and systems that aim to support cooperative work and argumentation processes. The first of these, gIBIS, provides a hypertext interface to the IBIS model. The tool has been used within MCC and in their client organizations to capture design-rationale information. gIBIS is a hypertext system intended to facilitate the capture and presentation of knowledge. However, as the underlying representation of the knowledge is not a formal one, gIBIS's ability to reason with that knowledge is severely restricted. Further, gIBIS also inherits the problems of IBIS—i.e., lack of explicit representation of goals and the outcomes of the argumentations.

IBE [39] is a hypertext system based on the IBIS model with functionalities similar to those of the gIBIS. A feature of IBE that distinguishes it from gIBIS is the integration of a document editor with hypertext. As with gIBIS, IBE is also constrained by the IBIS model and does not provide any reasoning mechanisms to use the captured knowledge. Discussing the need for future work, Lease *et al.* [39] state that argumentation network will be most useful only when the information is linked to artifacts (such as a specification, data flow diagram, source code, etc.) created in the process. Another functionality they advocate is the capture of the history of the development of artifacts to help the developer understand the thinking which went into their creation. Our research addresses both these concerns by providing two-way traceability between requirements and artifacts and facilities for temporal reasoning.

SIBYL is a system designed to *help users represent and manage the qualitative aspects of the decision making process - such as the alternative being considered, the goals to satisfy, and the arguments evaluating alternatives with respect to the goals* [19]. SIBYL is organized around decision graphs which record the pros and cons of choosing from a set of alternatives to satisfy a goal. Requirements in our model and goals in the DRL language of SIBYL represent similar constructs. Further, there is correspondence between IBIS primitives and DRL: issues correspond to the decision problem in DRL, positions correspond to alternatives, and arguments correspond to claims [40]. There are some differences between SIBYL's primitives and those of REMAP that probably derive from differences in the approach toward model development. Whereas the underlying model of SIBYL is motivated by the services it intends to provide—namely, viewpoint management, plausibility management, and dependency management—the REMAP model was developed in a task-specific context using an empirical study. The most tangible difference is the inclusion of primitives such as decisions, constraints, and design objects in REMAP. These “output oriented” primitives in REMAP

are explicitly related to the argumentation and requirements-refinement process.

Argnoter is a tool for discussing alternatives and selecting from among them [41]. Unlike REMAP, in Argnoter there is no explicit representation of goals or the problem being solved. The "pros" and "cons" in Argnoter correspond to "supports" and "objects to" relations in our model. The assumptions in our model are similar to the assumptions in Argnoter. Argnoter also has a notion of "belief sets" which map a set of assumptions into those of believed and not believed. This is similar to our mechanism for assigning belief values to arguments. The underlying model behind Argnoter can be seen as a special case of our conceptual model.

S. Toulmin proposed a model of Argumentation [42]. In it, a *claim* is the primary assertion being made. A *datum* is the object that supports the claim. The support provided by the *datum* to *claim* is qualified by *qualifier*. A *rebuttal* provides a reason for the qualifier. A *warrant* provides the basis on which a *claim* is supported by a *datum*. Finally, a *warrant* in turn is supported by a *backing*. Although the model provides several primitives for expressing justifications for an assertion, it does not provide primitives to represent the context in which such justifications are made. In our model, requirements and issues provide such a context. The role of *qualifier*, *rebuttal*, *warrant*, and *backing* can be supported by assumptions in our model. In this sense, our model provides a more uniform and extensible representation. Further, Toulmin's model is only a model for providing justification and does not relate the process to its outcomes.

SYNVIEW is a system developed for supporting cooperatively indexing, evaluating, and synthesizing information [43]. This system supports a structured debate in which conclusions are explicitly justified and negated by evidence. The underlying model of argumentation used is a simplified version of Toulmin's model. REMAP differs from this research in its formal representation of the dependencies. Further, in SYNVIEW there is no explicit representation of the context in which the argumentation occurs. Finally, the assumptions behind the evidence are not explicitly captured.

The CoNeX project [20], which also has adopted Toulmin's model, concentrates on integrating the semantics of the software development domain with group work. A related effort in the DAIDA project [44] is complementary to the REMAP effort in that it focuses on coordination aspects rather than the role of decision-oriented design rationale in software development.

Finally, another important approach to capturing design history is *process programming* [45]. This primarily uses algorithmic programming to describe the process of developing a program. Although such an approach is useful for representing how artifacts are created, it is not best suited for representing all the rationales behind design decisions—including the *why* of design decisions.

VI. DISCUSSION

We are currently investigating several potential extensions to REMAP. Although our model has been developed in the

context of the requirements-engineering task for a large-scale transaction-processing systems, its primitives are sufficiently generic to be applicable for all types of systems and across various phases of systems development. We are currently investigating the applicability of our model to all phases of the development of large-scale complex systems. To facilitate the capture of process knowledge with minimum overhead, the development of a design-support tool integrated with electronic mail is under study. Such an intelligent electronic mail system may use a structured communication protocol based on our model, natural language parsing technology, or act as an *information lens*. As our objective is to support various stakeholders, providing a comprehensive set of predefined queries which address various stakeholder needs will be very helpful. Further, an easy-to-use interface is a critical factor in ensuring the success of such a system. Finally, we propose to provide support for the design decision-making process with mechanisms such as multicriteria decision-support models, ranking, and voting mechanisms to facilitate the selection of an alternative from among many.

REFERENCES

- [1] C. Potts and G. Bruns, "Recording reasons for design decisions," in *Proc. 10th Int. Conf. on Software Eng.* (Singapore), Apr. 1988, pp. 418-426.
- [2] V. Dhar and M. Jarke, "Dependency directed reasoning and learning in systems maintenance support," *IEEE Trans. Software Eng.*, vol. 14, pp. 211-227, Feb. 1988.
- [3] B. Curtis, H. Krasner, and N. Iscoe, "A field study of the software design process for large systems," *Commun. ACM*, vol. 31, pp. 1268-1287, Nov. 1988.
- [4] H. Thimbleby, "Delaying commitment," *IEEE Software*, pp. 78-86, May 1988.
- [5] R. Guindon and B. Curtis, "Control of cognitive processes during software design: what tools would support software designers?" in *Proc. CHI '88* (Washington, DC), 1988, pp. 263-268.
- [6] C. Wild and K. Maly, "Toward a software maintenance support environment," in *Proc. Conf. on Software Maintenance* (Washington, DC), 1988, pp. 297-306.
- [7] R. Balzer, T. Cheatham, and C. Green, "Software technology in the 1990's: using a new paradigm," *IEEE Computer*, vol. 16, pp. 39-45, Nov. 1983.
- [8] R. Balzer, "A 15-year perspective on automatic programming," *IEEE Trans. Software Eng.*, vol. SE-11, pp. 1257-1267, Nov. 1985.
- [9] T. Biggerstaff and C. Richter, "Reusability framework, assessment, and directions," *IEEE Software*, pp. 41-49, July 1987.
- [10] R. V. Scoy and C. Plinta, "An informal experiment in reuse," in *Proc. Reuse in Practice Workshop* (Pittsburgh, PA), July 1989.
- [11] M. D. Lubars and B. Ramesh, "Domain analysis for multiple target systems," in *Proc. Workshop on Domain Modeling for Software Eng., OOPSLA '89* (New Orleans), Oct. 1989, pp. 92-96.
- [12] M. Shaw, *Group Dynamics: The Psychology of Small Group Behavior*. New York: McGraw-Hill, 1981.
- [13] B. Fisher, "Leadership: when does the difference make a difference?" in *Communication and Group Decision Making*, R. Hirokawa and M. Poole, Eds. Beverly Hills, CA: Sage, 1986.
- [14] D. Walz, J. Elam, H. Krasner, and B. Curtis, "A methodology for studying software design teams: An investigation of conflict behavior in the requirements definition phase," in *Proc. 2nd Workshop on Empirical Studies of Programmers*. Norwood, NJ: Ablex, 1987.
- [15] J. Conklin and M. Begeman, "glBIS: a hypertext tool for exploratory policy discussion," *ACM Trans. Office Inform. Syst.*, vol. 6, pp. 303-331, Oct. 1988.
- [16] K. B. Yakemovic and E. J. Conklin, "Report on a development project use of an issue-based information system," in *Proc. Conf. on Computer-Supported Cooperative Work*, Oct. 1990, pp. 105-118.
- [17] H. Rittel, "Dilemmas in a general theory of planning," *Policy Sci.*, vol. 4, pp. 155-169, 1973.
- [18] T. Winograd, "A language/action perspective on the design of cooperative work," *Human Computer Interaction*, vol. 3, no. 1, pp. 3-30,

- 1988.
- [19] J. Lee, "Sibyl: a qualitative decision management system," in *Artificial Intelligence at MIT: Expanding Frontiers*, P. Winston and S. Shellard, Eds. Cambridge, MA: MIT, 1990, ch. 5, pp. 106-133.
 - [20] U. Hahn, M. Jarke, and T. Rose, "Teamwork support in a knowledge-based information systems environment," *IEEE Trans. Software Eng.*, vol. 17, pp. 467-482, May 1991.
 - [21] R. Thayer and M. Thayer, "Glossary," in *System and Software Requirements Engineering*, R. Thayer and M. Dorfman, Eds. Los Alamitos, CA: IEEE Computer Soc. Press, 1990, pp. 605-676.
 - [22] L. Hubert, "Kappa revisited," *Psychological Bull.*, vol. 84, pp. 288-297, 1977.
 - [23] S. Biggs, "An Investigation of the decision processes underlying the assessment of corporate earning power," Ph.D. thesis, Univ. Minnesota, 1978.
 - [24] I. Vessey, "An investigation of the psychological processes underlying the debugging of computer programs," Ph.D. thesis, Dept. Commerce, Univ. Queensland (Australia), Apr. 1984.
 - [25] P. Johnson, F. Hasselbrock, A. Duran, and J. Moller, "Multimethod study of clinical judgment," *Organization. Behavior and Human Performance*, vol. 30, pp. 201-230, 1982.
 - [26] J. Mylopoulos, A. Borgida, M. Jarke, and M. Koubarakis, "Telos: representing knowledge about information systems," *ACM Trans. Inform. Syst.*, vol. 8, pp. 325-362, Oct. 1990.
 - [27] M. Jarke, M. Jeusfeld, and T. Rose, "A software process model for knowledge engineering in information systems," *Inform. Syst.*, vol. 15, no. 1, pp. 85-116, 1990.
 - [28] S. Greenspan, "Requirements modeling: a knowledge representation approach to software requirements definition," Ph.D. thesis, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 1984.
 - [29] J. Allen and P. Hayes, "Moments and points in an interval-based temporal logic," *Comput. Intell.*, vol. 5, pp. 225-238, Nov. 1989.
 - [30] R. Prieto-Diaz and P. Freeman, "Classifying software for reusability," *IEEE Software*, pp. 6-16, Jan. 1987.
 - [31] M. Lubars, "The idea design environment," in *Proc. 11th Int. Conf. on Software Eng.* (Pittsburgh, PA), 1989, pp. 23-32.
 - [32] M. Jarke, "Conceptbase v 3.0 user manual," Univ. Passau, Passau, Germany, Tech. Rep. MIP-9106, Mar. 1991.
 - [33] M. Lubars, "Code reusability in the large versus code reusability in the small," in *Software Reuse: Emerging Technology*, W. Tracz, Ed. Washington, DC: IEEE Computer Soc. Press, 1988, pp. 68-76.
 - [34] C. Rich and R. C. Waters, "Automatic programming: myths and prospects," *IEEE Computer*, pp. 40-51, Aug. 1988.
 - [35] H. Reubenstein and R. Waters, "The requirements apprentice: automated assistance for requirements acquisition," *IEEE Trans. Software Eng.*, vol. 17, pp. 226-240, Mar. 1991.
 - [36] D. R. Barstow, "Domain-specific automatic programming," *IEEE Trans. Software Eng.*, vol. SE-11, pp. 1321-1336, Nov. 1985.
 - [37] A. J. Czuchry and D. R. Harris, "KBRA: a new paradigm for requirements engineering," *IEEE Expert*, pp. 21-35, Winter 1988.
 - [38] D. R. Smith, "KIDS: a semiautomatic program development system," *IEEE Trans. Software Eng.*, vol. 16, pp. 1024-1043, Sept. 1990.
 - [39] M. Lease, M. Lively, and J. Leggett, "Using an issue-based hypertext system to capture the software life-cycle process," *Hypermedia*, vol. 2, no. 1, 1990.
 - [40] J. Lee, "SIBYL: a tool for managing group decision rationale," in *Proc. Conf. Computer-Supported Cooperative Work* (Los Angeles, CA), Oct. 1990, pp. 79-92.
 - [41] M. Stefik, D. Bobrow, S. Lanning, D. Tatar, and G. Foster, "WYSIWIS revised: early experiences with multi-user interfaces," *ACM Trans. Office Inform. Syst.*, vol. 5, pp. 147-167, Apr. 1987.
 - [42] S. Toulmin, R. Rieke, and A. Janik, *An introduction to Reasoning*, 2nd ed. New York: Macmillan, 1984.
 - [43] D. G. Lowe, "Co-operative structuring of information: the representation of reasoning and debate," *Int. J. Man-Machine Studies*, vol. 23, pp. 97-111, Aug. 1985.
 - [44] T. Rose, M. Jarke, M. Gocek, M. Maltzahn, and H. Nissen, "A decision based configuration process environment," *Software Eng. J.*, vol. 6, no. 5, pp. 332-346, 1991.
 - [45] L. Osterweil, "Software processes are software too," in *Proc. 9th Int. Conf. on Software Eng.* (Monterey, CA), Apr. 1987, pp. 2-13.



Balasubramaniam Ramesh (M '92) received the Masters degree in business administration in 1983 from the Indian Institute of Management, Calcutta, and the Ph.D. degree in information systems in 1992 from New York University.

He is Assistant Professor of Information Systems at the Naval Postgraduate School, Monterey, CA. His primary research is in the area of developing formalisms and models for knowledge-based support for information systems development. His research interests include methodologies for information systems design, computer-supported cooperative work, and decision-support systems. He is member of the ACM, AAAI, and TIMS.



Vasant Dhar is Associate Professor of Information Systems at the Leonard N. Stern School of Business at New York University. His research focuses on applying and developing knowledge representation and reasoning models for a variety of planning and design problems such as business process re-engineering, large software systems design and maintenance, pattern discovery from very large databases, financial hedge design, portfolio management, and audit planning. He has also been involved in research relating Artificial Intelligence and Operations Research modeling techniques; for example, rule/object-based expert systems and integer programming.

Professor Dhar is a member of the American Association for Artificial Intelligence, the Association for Computing Machinery, the Institute of Management Sciences, and the Society for Information Management.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.